École Supérieure en Informatique – Sidi-Bel-Abbès

3rd Year – Second Cycle – Option CyS

IA for Cybersecurity

# Lab 02 – Report

## Data Visualization and Analysis for a Cybersecurity Use Case

### Analysis of log data

Detecting a directory-enumeration scan
in a web-server access log

**Student:** Ahmed Mohamed Houssam

**Supervisor:** Dr R. Bousmaha

**Date:** 26 October 2026

**Tool:** Python (pandas, NumPy, Matplotlib, Seaborn)

**Deliverables:** `Lab02_Log_Analysis.ipynb` and this report

# Contents

**1 Introduction and objective** **2**

**2 Chosen use case** **2**

**3 Dataset** **2**

3.1 Why a simulated dataset . . . . . 2

3.2 Format . . . . . 3

3.3 Content of the simulation . . . . . 3

**4 Tool and methodology** **4**

**5 Exploration of the data** **4**

**6 The anomaly** **4**

**7 Visualizations** **5**

7.1 Figure 1 – When: volume and 404 errors over time . . . . . 5

7.2 Figure 2 – What: status codes by hour . . . . . 6

7.3 Figure 3 – Who: top sources of 404 errors . . . . . 6

**8 Detection rule** **7**

8.1 From log lines to windows . . . . . 7

8.2 The rule . . . . . 7

8.3 Justification of each decision . . . . . 8

**9 Evaluation** **8**

9.1 Protocol . . . . . 8

9.2 Results on anomalous data (day 1) . . . . . 9

9.3 Results on normal data (day 2) . . . . . 9

9.4 Why 404 and not request volume: comparison . . . . . 9

9.5 Sensitivity to the threshold . . . . . 10

**10 Main findings** **10**

**11 Limitations and improvements** **11**

**12 Conclusion** **11**

**A Reproducing the results** **12**

# 1 Introduction and objective

The objective of the lab is to apply data visualization and analysis techniques to one cybersecurity use case: select or simulate a dataset, explore it, inject or identify one anomaly, produce at least two visualizations that show it, define one simple detection rule, evaluate that rule on anomalous and normal data, and present the findings.

This report documents the work done in the accompanying Jupyter notebook `Lab02_Log_Analysis.ipynb`. Table 1 shows where each task of the lab sheet is answered.

| Task | Lab requirement | Where (report / notebook section) |
|---|---|---|
| 1 | Choose one use case | Section 2 / Section 1 |
| 2 | Select or simulate a dataset | Section 3 / Section 2 |
| 3 | Choose a tool | Section 4 / Section 3 |
| 4 | Explore and analyse the data | Section 5 / Section 4 |
| 5 | Inject or identify one anomaly | Section 6 / Section 5 |
| 6 | At least two visualizations | Section 7 / Section 6 (three figures) |
| 7 | One simple detection rule | Section 8 / Section 7 |
| 8 | Evaluate on anomalous and normal data | Section 9 / Section 8 |
| 9 | Present and explain the main findings | Sections 10–11 / Section 9 |

**Table 1:** Mapping between the lab tasks and this work.

# 2 Chosen use case

The chosen use case is **Analysis of log data**, applied to a web-server access log.

**Why this use case.**

- Logs are the raw material of every Security Operations Centre (SOC) and SIEM: the use case is easy to motivate and to defend.
- A web access log has a real, standard text format (Apache/Nginx "combined"). This allows the whole pipeline to be shown: raw text → parsing → table → visualization → rule.
- The anomaly has a clear and explainable signature, which makes a *simple* rule possible and meaningful.

**The attack studied: directory enumeration.** Before attacking a web site, an attacker usually does not know which pages exist. A tool such as `gobuster`, `dirb` or `ffuf` takes a wordlist (`admin`, `backup`, `.env`, ...) and requests every entry, hoping for an answer other than *404 Not Found*. Almost every guess fails, so the server log shows a sudden burst of 404 responses coming from one source address. The attack is a typical first step of web penetration tests and of real intrusions, which makes it a realistic target for a detection rule.

# 3 Dataset

## 3.1 Why a simulated dataset

The lab allows a selected *or* simulated dataset. A simulated log was chosen for four reasons:

1. **Ground truth.** We know exactly which requests belong to the attack, which is required for a proper evaluation (true/false positives and negatives). Public log datasets rarely provide request-level labels.

2. **Control over hard cases.** We can add harmless behaviours that look suspicious, to test whether the rule is really sound.
3. **Reproducibility.** Fixed random seeds give identical results at every run, on any machine, with no download.
4. **No privacy or legal issue.** All IP addresses are taken from the ranges reserved for documentation by RFC 5737 (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`); they never designate a real machine.

### 3.2 Format

Each file is plain text in the Apache/Nginx *combined* format, one line per request:

```
IP - - [timestamp] "METHOD /path PROTOCOL" status bytes "referer" "user-agent"
```

Example of an attack line (the notebook prints real examples):

```
198.51.100.23 - - [20/May/2026:03:12:25 +0000] "GET /admin.php HTTP/1.1" 404 196 "-"
                                                "gobuster/3.6"
```

### 3.3 Content of the simulation

Two 24-hour files are generated (Table 2).

| File | Lines | Content | Purpose |
|---|---|---|---|
| `access_day1.log` | 8 774 | normal traffic + attack | analysis, design of the rule |
| `access_day2.log` | 7 799 | normal traffic only (other seed) | held-out test of false alarms |
| `ground_truth_day1.csv` | 8 774 | label and actor per line | plotting and evaluation *only* |

**Table 2:** Files produced by the notebook (folder `data/`).

The labels are stored in a *separate* file and are never used by the detection rule, only to colour figures and to compute the evaluation.

Table 3 lists the simulated actors. Two of them are *hard negatives*: they are harmless but resemble the attack along one dimension, so they test the rule.

| Actor | Behaviour | Role in the study |
|---|---|---|
| About 80 visitors | Sessions of 2–7 page views, each followed by a few images/CSS/JS files; busy 09:00–17:00; about 2 % of page views hit a stale link (404) | Normal baseline |
| Crawler (`ExampleBot`) | 5 bursts of 70 requests in about one minute, roughly 3 % of them 404 | **Hard negative 1**: high volume, harmless |
| Uptime monitor | `GET /health` every 5 minutes | Periodic machine traffic |
| User on a broken page | A legacy page whose images are all missing: 12 (day 1) or 14 (day 2) 404 within seconds | **Hard negative 2**: a legitimate burst of 404 |
| **Attacker** (day 1 only) | `gobuster`-style scan at 03:12 | **Injected anomaly** |

**Table 3:** Actors of the simulation.

# 4 Tool and methodology

**Tool: Python** with pandas (parsing, grouping), NumPy (seeded generation), Matplotlib and Seaborn (figures). Python was preferred to Kibana because the full analysis fits in a single reproducible notebook, needs no server, and every step can be shown line by line.

**Pipeline.**

1. **Generate** the two logs (fixed seeds) and write them as text files.
2. **Parse** each line with a regular expression that follows the combined format; the number of unparsed lines is counted and must be zero, so that no data is silently lost.
3. **Explore** with summary tables (status codes, top IPs).
4. **Visualize** the anomaly from three different angles: when, what, who.
5. **Aggregate** into windows (one row per source IP and per minute) and plot the feature space.
6. **Define** one rule on a window counter and **evaluate** it on day 1 (labelled, with attack) and on day 2 (unseen normal day).

# 5 Exploration of the data

All 8 774 lines of day 1 and all 7 799 lines of day 2 were parsed (0 unparsed lines). Day 1 contains 83 distinct source IP addresses. Table 4 gives the status-code distribution of day 1.

| **Status** | 200 | 301 | 302 | 304 | 403 | 404 | 500 |
|---|---|---|---|---|---|---|---|
| **Requests** | 6 261 | 53 | 93 | 1 662 | 5 | 692 | 8 |

**Table 4:** HTTP status codes in day 1 (8 774 requests).

**Observations.**

- Most traffic is `200`/`304`: normal delivery of pages and files, following a human daily rhythm.
- By *volume*, the scan comes first (654 requests), but the second and third busiest sources are the harmless crawler (350) and the uptime monitor (288). Volume is a blunt signal: a threshold low enough to catch a slower scan also fires on harmless machines.
- By *404 errors*, one address stands far above all the others: 645 of the 692 errors of the day, whereas the next IPs have only 16 (crawler) and 12 (broken page). This is the lead followed in the rest of the study.

# 6 The anomaly

**Injected anomaly.** On day 1, at **03:12:25**, the address `198.51.100.23` starts a directory enumeration with user-agent `gobuster/3.6`. It tests 109 words, each with 6 extensions (none, `.php`, `.bak`, `.zip`, `.txt`, `.old`), that is 654 distinct paths. The attack therefore lasts 176 seconds (about 3.7 requests per second) and ends at 03:15:21. At night, when almost nobody uses the site, this is clearly abnormal.

**Where it occurs** (using the ground truth):

- time: 03:12:25 – 03:15:21 (four one-minute windows: 03:12, 03:13, 03:14, 03:15);

- source: `198.51.100.23`, one IP, one user-agent;
- size: 654 requests, of which 98.6 % (645) answered 404.

**What the attacker found.** Nine guesses did *not* return 404 (Table 5). This is what an analyst would have to review after the detection.

| Path | Status | Meaning |
|---|---|---|
| `/admin` | 302 | redirection: an administration area exists |
| `/login` | 200 | login page reachable |
| `/blog`, `/uploads` | 200, 301 | public / redirected resources |
| `/backup.zip`, `/.env`, `/server-status`, `/.htaccess`, `/.htpasswd` | 403 | the files *exist* but are forbidden: their existence is leaked |

**Table 5:** Non-404 answers received by the attacker.

# 7 Visualizations

Three figures show the anomaly from three complementary points of view: *when* (Figure 1), *what kind of response* (Figure 2) and *who* (Figure 3). A fourth figure (Figure 4) supports the choice of the detection rule.

## 7.1 Figure 1 – When: volume and 404 errors over time

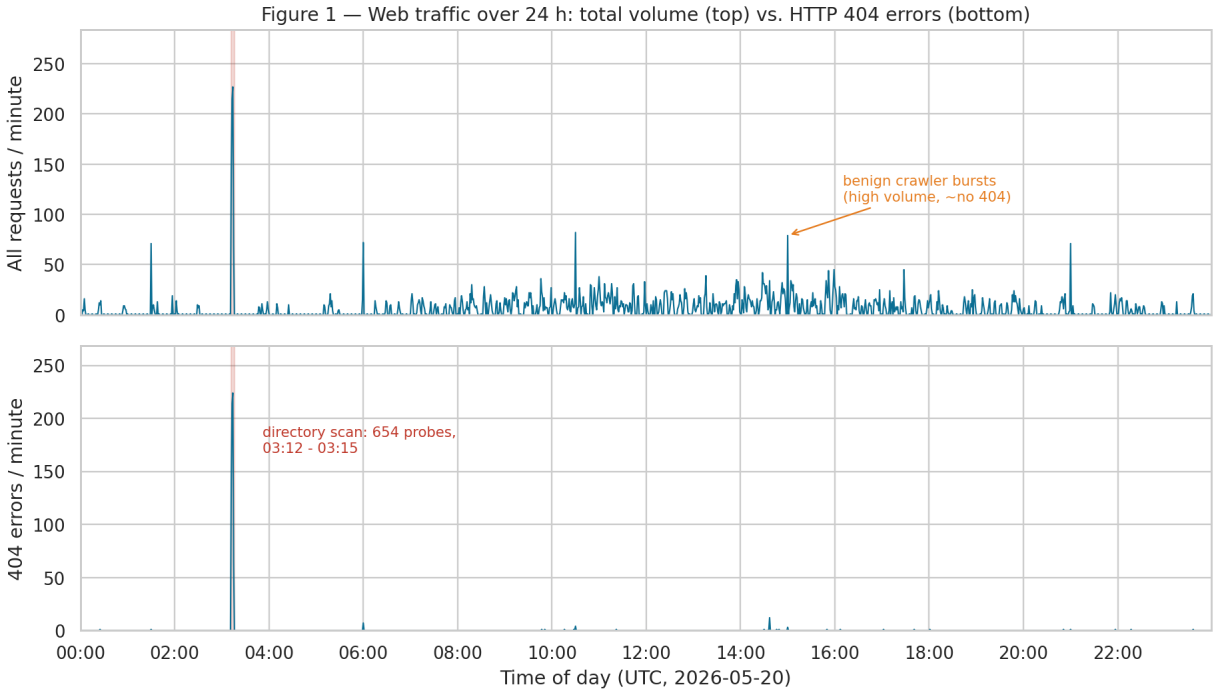



**Figure 1:** Requests per minute (top) and 404 errors per minute (bottom) over 24 hours. The red band marks the attack. The crawler also produces spikes in the top panel.

**Reading.** In the top panel the scan is the tallest spike, but the crawler and busy browsing periods also produce spikes: volume is a blunt signal. In the bottom panel only one event stands out: the 404 count is nearly flat (mostly 0–3 per minute, at most 12) except for the scan. *404 errors are a*

*much cleaner signal than total volume.* Empty minutes are filled with zero so that the line shows real silence.

## 7.2 Figure 2 – What: status codes by hour

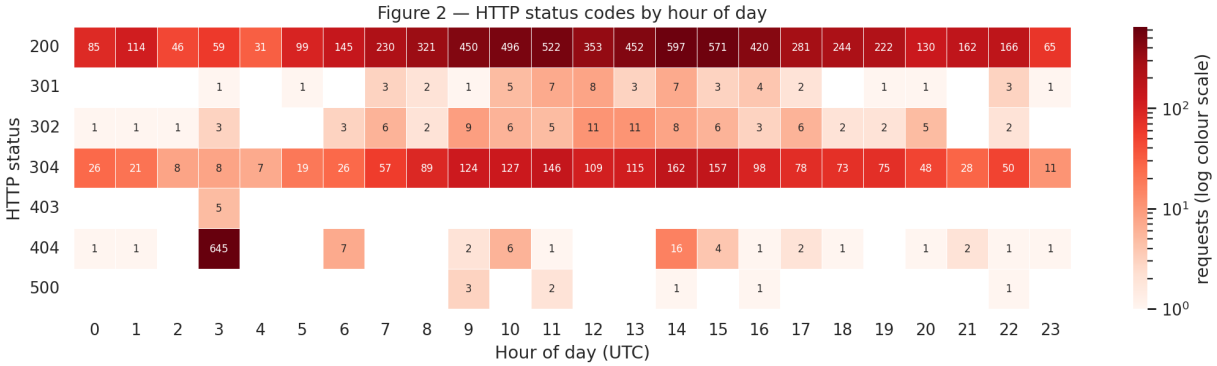



**Figure 2:** Heatmap of the number of requests per HTTP status code and per hour (logarithmic colour scale).

**Reading.** The 404 row is light for 23 hours and has one very dark cell at hour 3 (645 errors). The 403 row exists *only* at hour 3: those five answers are scan guesses that hit protected files. A logarithmic colour scale is used so that small counts stay readable next to the large one. The heatmap shows that the anomaly concerns *error responses at night*, not traffic in general.

## 7.3 Figure 3 – Who: top sources of 404 errors

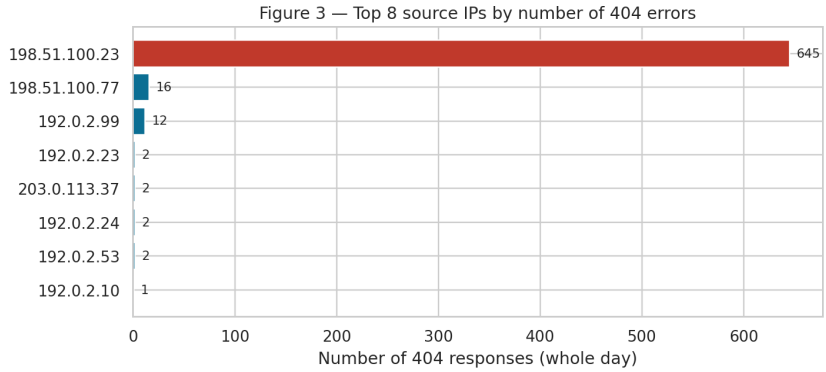



**Figure 3:** Top 8 source IPs by number of 404 responses (whole day).

**Reading.** A single address, `198.51.100.23`, produces 645 errors; all other addresses together produce 47. This identifies the *source* and justifies a rule computed *per IP*.

# 8 Detection rule

## 8.1 From log lines to windows

The log is aggregated into windows: one row per pair (source IP $i$, one-minute window $m$) with two counters,

$n_{\text{req}}(i, m)$ = number of requests, $n_{404}(i, m)$ = number of requests answered with status 404.

Day 1 gives 1 262 windows and day 2 gives 1 216 windows. Figure 4 plots every window of day 1.

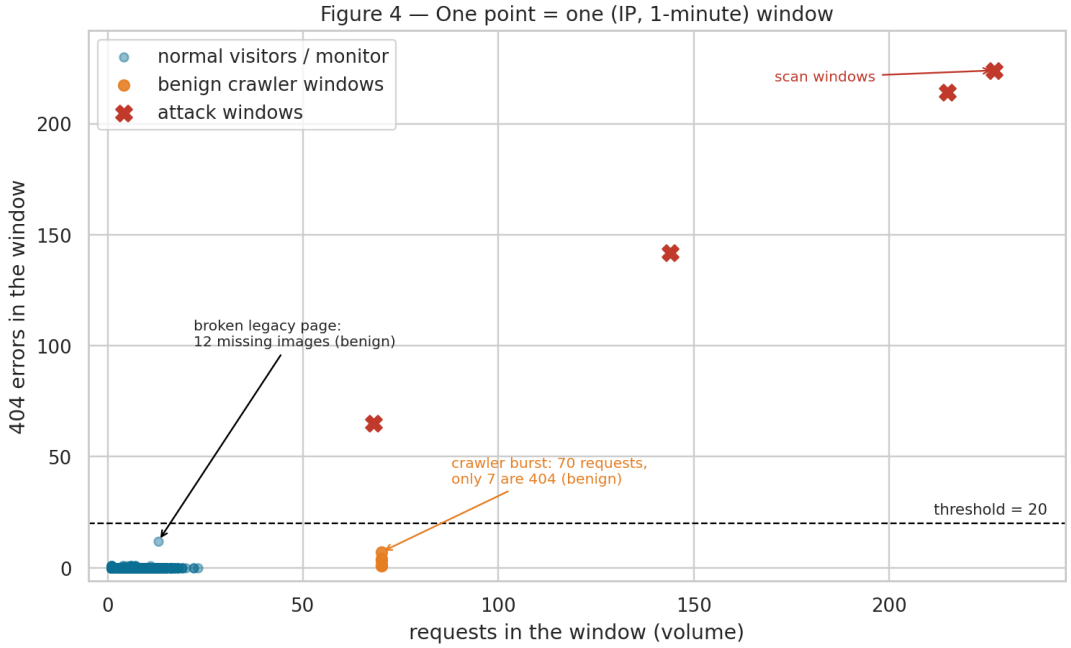



**Figure 4:** Feature space: one point per (IP, 1-minute) window. The dashed line is the threshold of the rule.

**Reading.**

- **Blue** (normal): few requests and few 404, in the bottom-left corner. The highest normal point is the broken legacy page (12 errors).
- **Orange** (crawler): 70 requests per window, but at most 7 are 404. A rule based on volume would flag it; a rule based on 404 does not.
- **Red** (scan): 65 to 224 errors per minute, clearly above everything else.

## 8.2 The rule

**Raise an alert if one source IP receives more than 20 HTTP 404 responses within a one-minute window.**

$$\text{alert}(i, m) \iff n_{404}(i, m) > T, \qquad T = 20$$

### 8.3 Justification of each decision

| Decision | Justification |
|---|---|
| Feature = number of **404** | It directly encodes the behaviour of the attack (guessing paths that do not exist) and ignores harmless volume. Section 9 shows by experiment that a volume rule is worse. |
| Counted **per IP** | An attack comes from a source; per-IP counting avoids mixing it with the crowd and gives the analyst an address to block. |
| **1-minute** window | Short enough to react quickly, long enough to count a burst; simple to explain and to implement in a SIEM. |
| Threshold $T = \mathbf{20}$ | It is above the largest normal value observed (12 on day 1, 14 on day 2) and far below the weakest scan minute (65). The sweep in Section 9 shows that every value from 12 to 64 gives a perfect result on day 1, so $T = 20$ does not depend on a fragile choice. |
| A rule, not a ML model | The lab asks for one *simple* rule. A rule is transparent, explainable to an analyst, needs no training data and is easy to defend. |

**Table 6:** Design decisions of the rule.

On day 1 the rule raises exactly four alerts, one for each attack minute (Table 7).

| Source IP | Window | $n_{\text{req}}$ | $n_{\mathbf{404}}$ |
|---|---|---|---|
| 198.51.100.23 | 2026-05-20 03:12 | 144 | 142 |
| 198.51.100.23 | 2026-05-20 03:13 | 215 | 214 |
| 198.51.100.23 | 2026-05-20 03:14 | 227 | 224 |
| 198.51.100.23 | 2026-05-20 03:15 | 68 | 65 |

**Table 7:** Alerts raised on day 1.

## 9 Evaluation

### 9.1 Protocol

The rule is evaluated on both kinds of data, as required.

- **A – Anomalous data (day 1).** A window is *positive* if it contains at least one request of the attack, *negative* otherwise (labels from the ground truth). Definitions: TP = attack window with alert; FN = attack window without alert; FP = normal window with alert; TN = normal window without alert.
- **B – Normal data (day 2).** A different normal day, generated with another seed and not used to choose the threshold. Any alert is a false alarm.

Metrics: precision = $\frac{TP}{TP+FP}$ (how many alerts are real), recall = $\frac{TP}{TP+FN}$ (how many attack windows are caught), false-positive rate = $\frac{FP}{FP+TN}$, and $F_1$, the harmonic mean of precision and recall.

### 9.2 Results on anomalous data (day 1)

| | Alert | No alert |
|---|---|---|
| **Attack window** (4) | TP = 4 | FN = 0 |
| **Normal window** (1 258) | FP = 0 | TN = 1 258 |

| Metric | Value |
|---|---|
| Precision | 1.000 |
| Recall | 1.000 |
| False-positive rate | 0.000 |
| $F_1$ | 1.000 |

**Table 8:** Day 1, window level (threshold 20).

At the level of IP addresses (what an analyst cares about), among the 83 addresses of day 1 the rule flags exactly one, the attacker: TP = 1, FP = 0, FN = 0, TN = 82. The first alert is available 35 seconds after the scan started (03:12:25), because the rule is evaluated when the 03:12 window closes at 03:13:00.

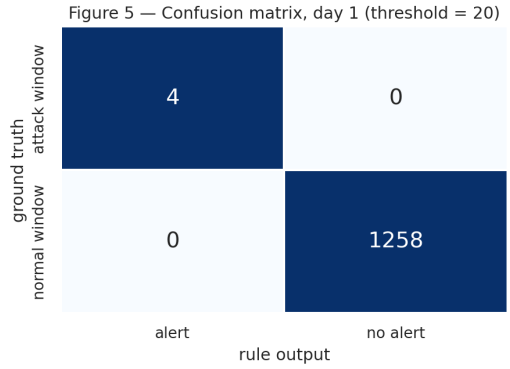



**Figure 5:** Confusion matrix on day 1 (colour = share inside each real class, numbers = counts).

### 9.3 Results on normal data (day 2)

Day 2 contains 7 799 requests and 1 216 windows. The largest 404 count in any window is 14 (the broken legacy page of that day), below the threshold. The rule raises **0 alerts**: no false alarm, including on the two hard negatives (crawler and broken page).

### 9.4 Why 404 and not request volume: comparison

To justify the central design decision, the same threshold (20) is applied to two features on day 1.

| Rule | TP | FP | FN | TN | Prec. | Recall | $F_1$ |
|---|---|---|---|---|---|---|---|
| A: requests per minute > 20 | 4 | 8 | 0 | 1 250 | 0.333 | 1.000 | 0.500 |
| B: 404 per minute > 20 (chosen) | 4 | 0 | 0 | 1 258 | 1.000 | 1.000 | 1.000 |

**Table 9:** Volume rule versus 404 rule on day 1.

Both rules catch the attack, but the volume rule generates 8 false alarms (5 from the harmless crawler and 3 from busy visitors), so two alerts out of three would be wasted analyst time. Only the 404 feature separates the attack from harmless busy sources.

### 9.5 Sensitivity to the threshold

A good rule must not depend on a lucky number. The threshold was swept from 0 to 300 (Figure 6).

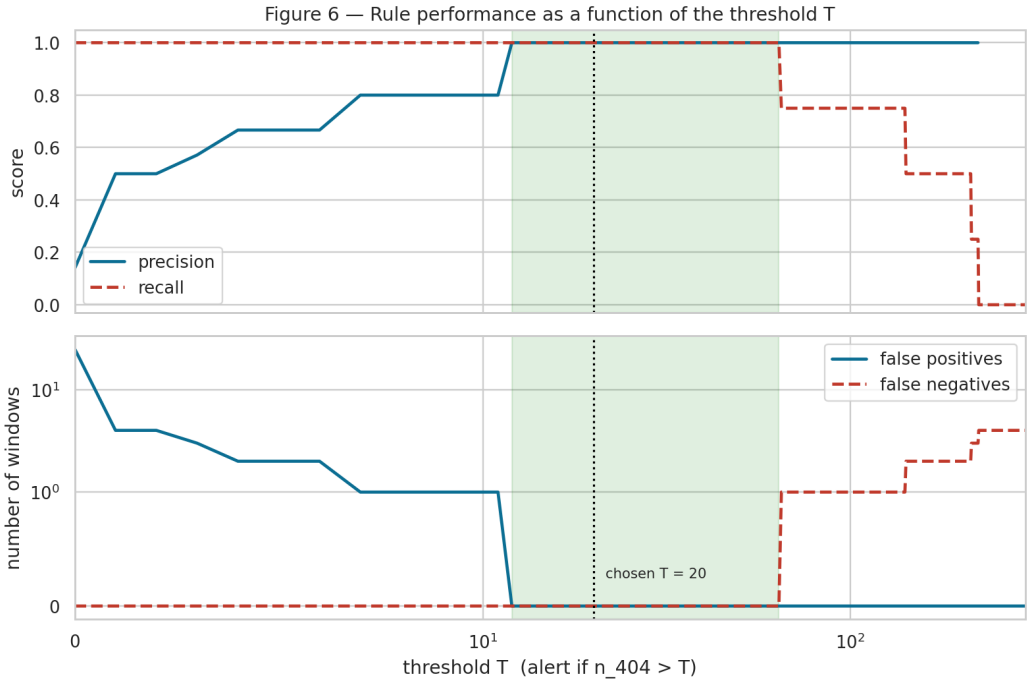



**Figure 6:** Precision, recall and error counts as a function of the threshold $T$. Green band: values giving no error. Dotted line: chosen value.

- Largest 404 count in a normal window: **12**. Smallest in an attack window: **65**.
- Every threshold from $T = 12$ to $T = 64$ gives zero false positives and zero false negatives; the chosen $T = 20$ lies inside this wide band.
- Left of the band the threshold is too low and normal windows (e.g. the broken page) trigger false alarms: precision falls. Right of the band the threshold exceeds some scan minutes and the attack is progressively missed: the weakest (last, partial) minute is lost first and recall reaches 0 above the strongest minute.

**Honest remark.** These perfect scores are obtained on data that we designed. They prove that the *logic* of the rule is sound, not that the same numbers would hold on a real server. On real logs, the threshold must be calibrated on a historical baseline (for instance the 99.9th percentile of $n_{404}$ per IP per minute).

## 10 Main findings

1. **The anomaly is visible and localisable.** A burst of about three minutes at night, from one IP, 654 requests, 98.6 % answered 404. It is the only event of the day that stands out in the 404 time series (Fig. 1), in the status-by-hour heatmap (Fig. 2) and in the ranking of sources (Fig. 3).
2. **Choosing the right feature matters more than choosing the right threshold.** Total volume is a blunt signal; the number of 404 per IP per minute separates the scan from harmless busy sources (Fig. 4, comparison table).

3. **The rule is simple and explainable:** one counter, one window, one threshold. On day 1 it catches all four attack windows with no false alarm among the 1 258 normal windows, and on the independent normal day 2 it raises no alert in 1 216 windows.
4. **The rule is robust to the exact threshold:** any value from 12 to 64 gives a perfect result.
5. **Detection is not the end of the work.** The log shows that the attacker discovered `/admin` and `/login` and learned that `/backup.zip`, `/.env`, `/server-status`, `/.htaccess` and `/.htpasswd` exist (403). A response would block the IP, check whether the administration pages should be publicly reachable, and confirm that the forbidden files are truly protected (ideally not stored in the web root).

## 11 Limitations and improvements

**Limitations.**

- **Simulated data**: the threshold must be re-calibrated on real traffic.
- **Low-and-slow scans**: an attacker sending fewer than 20 wrong paths per minute evades the rule. Countermeasure: a second rule on a longer window (e.g. more than 100 per hour).
- **Fixed window boundaries**: a burst split across two minutes (e.g. 15 + 15) escapes a tumbling window. Countermeasure: a sliding window.
- **Source-IP assumption**: a distributed scan (many IPs, few requests each) or many users behind one shared NAT address breaks the assumption "one IP = one actor".
- **User-agent not used on purpose**: `gobuster/3.6` is trivial to spoof, so a rule based on it would be easy to bypass.

**Possible improvements.** Add the number of *distinct* 404 paths and the 404 *ratio* as features; learn a per-IP baseline; apply an anomaly-detection model (Z-score, Isolation Forest) to the window table; forward the alert to a SIEM and trigger an automatic block (e.g. `fail2ban`).

## 12 Conclusion

A web-server access log was simulated in the real combined format, with an injected directory-enumeration scan and two harmless look-alikes (a crawler and a broken page). Three visualizations show the anomaly in time, by response type and by source. A single, transparent rule – more than 20 HTTP 404 responses from one IP in one minute – detects all attack windows, raises no false alarm on the normal windows of the same day nor on an independent normal day, and clearly outperforms a naive volume-based rule. The study also states honestly what the rule cannot detect and how it could be extended.

# A Reproducing the results

- Requirements: Python 3 and the packages listed in the file `requirements.txt`: numpy, pandas, matplotlib, seaborn, jupyter.
- Open the notebook and choose *Kernel* → *Restart & Run All* (no internet needed). It creates `data/` (logs, ground truth, results file) and `figures/` (the six figures of this report).
- All random generators use fixed seeds (2026 for day 1, 7 for day 2, 99 for the attack), so every number in this report is reproduced exactly.